Glitch Hunter — Tech Stack

Sep 29, 2026 · @Tanmay Singh

What this is

Glitch Hunter is a PPO reinforcement-learning agent that plays a Super Mario Bros clone and is watched live through a web dashboard; a second pipeline turns anything it catches breaking the game's own rules into an evidence bundle (screenshot, GIF, Markdown/PDF report). The table below lists every technology in the project and exactly which part of it uses that technology.

Stack by objective

Each objective built on the one before it: nothing was replaced, only added.

Objective	What it produced	Technologies it added
1 — Autonomous exploration agent	the 6M brain (46.8% level completion)	Python 3.12, Pygame, Gymnasium, PyTorch, Stable-Baselines3 PPO (CnnPolicy), NumPy, cloudpickle, Flask + Flask-SocketIO dashboard, OpenCV frame stream, HTML/CSS/JS, pytest, Git/GitHub
2 — QA / glitch-hunting agent	the 16M QA brain (84.25% coverage, 56.6% completion)	shared-memory coverage bitmap (NumPy + multiprocessing), reachability mask (.npz), QA reward and lifecycle, anchor-KL consolidation callback, 500-episode evaluation (multiprocessing Pool), SHA-256 artifact manifest, ruff, mypy, pytest-cov, GitHub Actions, uv
3 — Automated bug reporting	incidents with evidence, GIF/MD/PDF reports, replay; 6 benchmark bugs, all detected; a clean run that reaches the castle gets a "No Bugs Found" run report	incident pipeline and store, Pillow (GIF), fpdf2 (PDF), subprocess replay, zip bundles, REST API + Socket.IO events (incl. live status), a four-tab dashboard (Overview, Live Testing, How It Works, Bug History), a project-facts reader (dashboard/facts.py), game variants (mario_clean / mario_bugged), collision and jump-physics detectors, level-design reference file

Language, runtime and environment

Technology	Version	What it is	Where it is used	Objectives
Python	3.12 (exactly)	the programming language	the whole codebase: game wrapper, training, dashboard, reporting, tools, tests	1, 2, 3

Technology	Version	What it is	Where it is used	Objectives
venv (`venv_gpu/`)	—	isolated Python environment	holds every dependency; `run_dashboard.bat` and all commands use `venv_gpu\Scripts\python.exe`	1, 2, 3
pip + `requirements.txt`	—	package installer	the 2-step manual install (torch first, then `requirements.txt`)	1, 2, 3
uv + `uv.lock`	—	fast, locked installer	one-command install (`uv sync`) that encodes the torch/CUDA rules	2, 3
NVIDIA CUDA 12.6	cu126 build	GPU computing	fast PPO training on the GPU (CPU-only also works, just slower)	1, 2
Windows / Linux / macOS	Windows 11; Ubuntu and macOS (CI)	operating systems	development and dashboard on Windows; CI tests on all three	1, 2, 3
Python standard library	3.12	built-in modules	`multiprocessing` (training workers, shared memory, parallel evaluation), `threading` / `queue` (dashboard game thread, report worker), `subprocess` (incident replay, git, the dashboard's background test count), `hashlib` (SHA-256), `json`, `zipfile`, `difflib` (bug-diff pin), `argparse` (tools), `ctypes` (Windows window control, power settings)	1, 2, 3

Game engine and simulation

Technology	Version	What it is	Where it is used	Objectives
Pygame	2.6.1	2D game library (on SDL 2.28)	runs the Super Mario Bros clone: drawing, sprites, collisions, the game window	1, 2, 3
Mario clone (`games/mario_clean/`)	vendored, pinned by SHA-256	the game itself (Level 1-1)	the untouched baseline every brain is trained and measured on	1, 2, 3
`games/mario_bugged/`	copy of the clone	the game with 6 declared benchmark bugs	bug-detection testing; chosen on the dashboard's Live Testing tab	3
Gymnasium	1.3.0	standard RL environment API	`custom_mario_env.py` turns the game into an environment: 10 discrete actions, `step()` / `reset()`, info dict	1, 2, 3

Technology	Version	What it is	Where it is used	Objectives
Gymnasium wrappers	1.3.0	observation pipeline	frame skip 4, grayscale, resize to 84×84, stack of 4 frames (`wrap_observation`); `TimeLimit` in evaluation	1, 2, 3
SDL dummy video driver	—	headless rendering	training workers, evaluation, tests and incident replay run with no window	1, 2, 3
Windows `user32` via ctypes	—	native window control	`dashboard/game_window.py`: centre, show, hide and minimise the dashboard's game window to the taskbar; `dashboard/desktop.py`: centre the terminal and browser windows, and keep full speed on battery (power mode, process throttling), and stop the dashboard when Esc is pressed in its terminal	1, 3

Reinforcement learning and machine learning

Technology	Version	What it is	Where it is used	Objectives
PyTorch	2.14.0 (+cu126)	deep-learning framework	the neural network inside the agent; runs on the GPU in training, CPU in evaluation	1, 2, 3
Stable-Baselines3 — PPO	2.9.0	RL algorithm library	trains the agent (`train_agent.py`) and loads the brain for the dashboard and evaluation; the dashboard draws each move from the loaded policy, so every run takes a new route, on both games; a stalled run is taken over with running jumps and run-ups, and Mario is walked up to and jumped across the one pit he could never leave, so no run ends stuck	1, 2, 3
CnnPolicy	SB3	convolutional policy + value network	reads the 84×84×4 frames and chooses one of 10 actions	1, 2, 3
SubprocVecEnv / DummyVecEnv	SB3	vectorised environments	8 parallel game workers in training; a single env in the dashboard	1, 2
SB3 callbacks (custom)	project	hooks into training	checkpoints with coverage, anchor-KL consolidation, stagnation watch, value warm-up (`training/callbacks.py`)	1, 2
Anchor-KL consolidation	project	keeps a skill from being forgotten	after each update, pulls the policy back toward a frozen healthy brain if KL > 0.08	2
cloudpickle	3.1.2	serialisation	saving/loading checkpoints and passing objects to worker processes	1, 2, 3

Technology	Version	What it is	Where it is used	Objectives
TensorBoard (optional)	≥ 2.18	training-curve viewer	only if present; training runs fine without it	1, 2
The brains	—	trained models	`mario_brain_checkpoint.zip` (6M, Objective 1) and `glitch_hunter_main_brain.zip` (16M, Objectives 2–3), frozen and hash-checked	1, 2, 3

Numerical computing, data and storage

Technology	Version	What it is	Where it is used	Objectives
NumPy	2.5.2	arrays and maths	observations, the coverage bitmap, the testable mask, jump arcs, evaluation statistics	1, 2, 3
`.npz` files	NumPy	compressed array files	coverage state, reachability mask, jump arcs, anchor states (`exploration_data/`), loaded with pickle disabled	2
Shared memory	stdlib `multiprocessing.shared_memory`	memory shared between processes	one global coverage map written by all 8 training workers	2
JSON	stdlib	structured records	incident records, manifests, evaluation results, `artifacts.json`, `INJECTED_BUGS.json`, `level1_design.json`	1, 2, 3
JSON Lines (`.jsonl`)	stdlib	append-only logs	coverage audit trail, reward telemetry, incident occurrences, capture failures	2, 3
SHA-256 (hashlib)	stdlib	file fingerprints	`artifacts.json` protects the brains and masks; incidents and game variants are identified by hash	2, 3
ZIP	stdlib `zipfile`	archives	model checkpoints (SB3 format), incident context frames, the "Download all" evidence bundle	1, 2, 3

Exploration and QA logic (the project's own components)

Built in-house on top of the libraries above.

Component	Location	What it does	Objectives
Game environment	`custom_mario_env.py`	drives the game one frame at a time, reports Mario's position, speed, deaths and glitches	1, 2, 3
Reward wrapper	`agent_logic.py`, `rewards/legacy.py`, `rewards/qa.py`	completion reward (Objective 1); coverage + phase-gated QA reward (Objective 2)	1, 2
Spatial coverage	`exploration/coverage.py`	1-px world bitmap of everywhere Mario has been, persistent across episodes	2
Reachability / testable mask	`exploration/reachability.py`	which pixels Mario can physically reach (3 methods + real jump arcs) — the coverage denominator	2
Episode lifecycle	`exploration/lifecycle.py`	EXPLORE → COMPLETE only on evidence; safety reset on real stagnation, never on time	2
Completion evaluation	`evaluation/completion.py`	the frozen 500-episode protocol that decides HEALTHY / WARNING / REGRESSED	1, 2
Game variants	`reporting/variants.py`	identifies clean vs bugged game by tree hash; every bug change must be declared and marked	3
Collision and jump-physics detectors	`reporting/collision_invariants.py`	generic rules: clipping into solids, blocked by nothing, hit or stomp without contact, impossible jump	3
Level design reference	`reporting/level_design.py`, `level1_design.json`	the drawn ground, pipes and steps the detectors compare against	3

Image and video processing

Technology	Version	What it is	Where it is used	Objectives
OpenCV (opencv-python)	5.0.0.93	image processing	resizes each game frame to 480×360 and JPEG-encodes it for the live stream; PNG-encodes the exact trigger frame of a bug	1, 3

Technology	Version	What it is	Where it is used	Objectives
Pillow	12.3.0	image library	builds the context GIF of each incident and the finish GIF of each clean run, and prepares images for the PDF report	3
Pygame surfaces	2.6.1	raw frame capture	the full-resolution frame copied at the exact moment a detector fires	3

Bug evidence and reporting

Technology / component	Version	What it is	Where it is used	Objectives
fpdf2	2.8.8	PDF generator	the PDF bug report of every incident (`reporting/render.py`) and the run report of every clean run that reaches the castle (`reporting/run_report.py`)	3
Markdown	—	text report format	the Markdown bug report of every incident, and of every clean-run report	3
Incident pipeline	`reporting/pipeline.py`	capture → store → render → replay, in the background	turns each detection into a complete incident; retries and recovers after a crash	3
Incident store	`reporting/store.py`	evidence on disk	one read-only folder per incident with a hash manifest; atomic writes; never overwritten	3
Clean-run report	`reporting/run_report.py`	the "No Bugs Found" report	written when a clean-game run reaches the castle: run record, final frame, GIF, Markdown, PDF, downloadable as one zip	3
Incident schema	`reporting/schema.py`	the standard bug record	ids, timestamps, deterministic JSON, validation	3
Severity, confidence, duplicates	`reporting/analysis.py`	classification	severity per bug type, detector confidence, same-bug matching and occurrence counts	3
Replay (reproduction)	`reporting/reproduce.py` + subprocess	re-runs the episode headless	proves the bug happens again on the same frame: reproduced / not reproduced	3
Provenance	`reporting/provenance.py`	identity of brain, game and code	every report says which brain and which game produced it	3
Maintenance tool	`tools/incidents.py`	command line	list, show, verify, re-render, re-run replay	3

Web dashboard: backend

Technology / component	Version	What it is	Where it is used	Objectives
Flask	3.1.3	web framework	`app.py`: serves the page and the REST API: `/api/status` (state + live status), `/api/project` (the project's facts), `/api/incidents` (session, or `?all=1`), `/api/incidents/<id>`, `/api/runs` (every saved run report), `/healthz`, /api/stop (sent by Esc, then Yes, on the page; stops the dashboard like Ctrl+C), and the evidence and run-report downloads	1, 2, 3
Flask-SocketIO	5.6.1	real-time messaging	pushes video frames, log lines, the live status (10 times a second), bug-found, run-finished and report-ready events; receives start, stop, reset, game switch and page-opened (on python-socketio 5.17.0)	1, 2, 3
Threading mode	Flask-SocketIO	concurrency model	real OS threads (eventlet deliberately not used)	1, 2, 3
Werkzeug server	3.1.8 (via Flask)	local HTTP server	listens on 127.0.0.1:5000 only by default; no debug console	1, 2, 3
Game thread service	`dashboard/service.py`	one thread owns the game window	Start / Pause / Resume / Reset / game switch, 30 steps a second, pause on every bug, stop at the end of a clean-game run, forwards the live status every third step; a page refresh is a Reset	1, 3
Dashboard backend	`dashboard/backend.py`	env + brain + frame stream	loads the approved brain (hash-checked), plays, captures incidents, writes clean-run reports, and sends each step's live status: AI mode (EXPLORE / COMPLETE), action, run and step, level progress, coverage of the reachable level (the brain's own map plus new pixels); counts the brain's parameters	1, 2, 3

Technology / component	Version	What it is	Where it is used	Objectives
Project facts	`dashboard/facts.py`	the Overview page's numbers	`/api/project`: reads `FINAL_OBJECTIVE2.json`, the 6M baseline, `INJECTED_BUGS.json`, the detector list, the incident and run-report stores, git, and pytest's own test count (collected once, in a low-priority background process); anything it cannot read is left out, never guessed	3
Safe file serving	`reporting/store.py` and reporting/run_report.py allow-lists	security	only known incident / run-report ids and file names can be downloaded; path tricks get 404; `/api/runs` and `/api/project` only read	3

Web dashboard: frontend

Technology	Version	What it is	Where it is used	Objectives
HTML5 (Jinja template)	Jinja2 3.1.6	page structure	`dashboard/web/templates/index.html`: four tabs (Overview, Live Testing, How It Works, Bug History), game chooser, controls with an "i" button for the 10 actions, live video (NO SIGNAL until testing starts), Live status with its five-step progress list, Bug Tracker, detail windows (dialog elements), and the Esc "Stop the dashboard?" question	1, 2, 3
CSS3	—	styling	`dashboard/web/static/css/style.css`: retro arcade theme (black and green, a pixel font, CRT screen lines on the background (the game screen stays clean), a green-screen log terminal, small animations) with colours that carry meaning (running, healthy, warning, bug), responsive grid; the font is bundled, so it works offline	1, 2, 3

Technology	Version	What it is	Where it is used	Objectives
Vanilla JavaScript	ES2017+	page behaviour, no framework	`dashboard/web/static/js/main.js`: shows only what the server reports: tabs, controls, live video, Live status with its five-step progress list, the Bug found (with where to fix it) and run-result cards, the game screen holding its last frame when testing stops, Bug Tracker, Bug History tables with Delete and Clear history (asked first), detail windows, reset on page refresh, game switch, Esc to stop (asked first); inline SVG icons	1, 2, 3
Socket.IO JS client	vendored (`dashboard/web/static/vendor/`)	browser side of the live connection	receives frames and events; kept locally so the dashboard works offline	1, 2, 3
Press Start 2P font	vendored (`dashboard/web/static/vendor/fonts/`)	pixel font for headings and labels	the retro arcade look; kept locally so the dashboard works offline (SIL Open Font License 1.1)	3
Fetch API	browser	REST calls	loads `/api/status`, `/api/incidents`, `/api/project` and `/api/runs`	3

Testing, code quality and CI/CD

Technology	Version	What it is	Where it is used	Objectives
pytest	9.1.1	test framework	948 automated tests in `tests/` (fast ones by default, `-m slow` for brain and end-to-end)	1, 2, 3
pytest-cov	7.1.0	code coverage	the full gate requires at least 90% (last run 90.68%, on 2026-09-28, measured to two decimals)	2, 3
Ruff	0.16.6	linter	style, bugs and security (bandit) rules on every run	2, 3
mypy	2.3.1	static type checker	strict mode, checked for both Windows and Linux	2, 3
pre-commit	4.6.2	git hooks	runs checks before each commit	2, 3
Quality gate	`tools/check.py`	one command for all checks	lint → types → tests → artifact hashes (`--full` adds slow tests + coverage floor)	2, 3

Technology	Version	What it is	Where it is used	Objectives
Artifact verification	`tools/verify_artifacts.py`	integrity check	re-hashes the protected brains, masks and baseline against `artifacts.json`	2, 3
End-to-end validation	`tools/validate_incident_pipeline.py`	37-check pipeline proof	runs the real dashboard stack headless and checks every reporting stage	3
Git + GitHub	—	version control and hosting	source history and the remote repository	1, 2, 3
GitHub Actions	`.github/workflows/ci.yml`	continuous integration	lint, types and tests (86% coverage floor) on Ubuntu, Windows and macOS on every push, and the one-click setup on clean Windows, Linux and macOS machines, ending with the dashboard up on the approved brain; on Linux, a browser test of the dashboard	2, 3
Playwright	1.63.0	browser automation	drives the dashboard in headless Chromium in CI (`tests/test_dashboard_e2e.py`); installed only there	3

Tooling, scripts and documentation

Tool / format	Where it is used	Objectives
`train_agent.py`	training, with safe resume (`--resume-from`, `--dry-run-resume`)	1, 2
`tools/evaluate_completion.py`	500-episode completion test (HEALTHY / WARNING / REGRESSED)	1, 2
`tools/build_reachability.py`, `collect_jump_arcs.py`, `bootstrap_coverage.py`, `migrate_coverage.py`, `build_anchor_set.py`, `calibrate_*.py`	one-time setup and measurement for the QA campaign	2
`tools/build_level_design.py`	writes the level-design reference from the clean game	3
`run_dashboard.bat`	double-click to start the dashboard and open the browser	3
Markdown docs	`README.md`, `docs/OBJECTIVE1-3.md`, `docs/ARCHITECTURE.md`, `docs/PERFORMANCE.md`, `SECURITY.md`, `CONTRIBUTING.md`	1, 2, 3
Mermaid diagrams	the architecture diagram in `docs/ARCHITECTURE.md`	2, 3

Project map: every folder and file

Every folder and file in the project, tracked in git or not, with what it holds and what uses it. "In git" says whether the file is committed; gitignored files exist only on the machine that made or installed them. Large generated sets (the Python environment, caches) are described as a folder.

Folders at a glance

Folder	What it holds	Where it is used	In git
`common/`	small shared helpers: logging, atomic file writes, hashes, tool start-up	every entry point and tool	yes
`rewards/`	the two reward functions: completion (Objective 1) and QA exploration (Objective 2)	`agent_logic.py` during training and play	yes
`training/`	Stable-Baselines3 callbacks, checkpoint helpers, the one-off value-head reset	`train_agent.py`	yes
`evaluation/`	the 500-episode completion protocol, Level-1 verification, the frozen 6M baseline	training resume checks, `tools/evaluate_completion.py`, `tools/verify_level1.py`	yes (`results/` is not)
`exploration/`	`config.py` (every tuned value), spatial coverage, reachability mask, episode lifecycle, level completion	the environment, reward wrapper, training, dashboard	yes
`reporting/`	Objective 3: detectors, incident pipeline, store, reports, replay, run reports, game variants	dashboard, tests, `tools/incidents.py`	yes
`dashboard/` (`web/`)	the dashboard's own code (the game thread, the backend, the Overview facts, the desktop and game-window helpers) and, in web/, the page: HTML, CSS, JavaScript, bundled Socket.IO client and pixel font	served by `app.py`	yes
`docs/`	Objective 1-3 summaries, architecture, performance notes, contributing, security, third-party notices, the Tech Stack and Panel Q&A PDFs, and the README images (assets/)	readers; linked from `README.md`	yes
`tools/`	command-line tools: quality gate, evaluation, data builders, incident tools, installer	run by hand and by `tools/check.py` / tests	yes

Folder	What it holds	Where it is used	In git
`tests/`	948 automated tests and their helpers	`pytest`, `tools/check.py`, CI	yes
`.github/`	the CI workflow	GitHub Actions on every push	yes
`games/mario_clean/`	the original Mario clone (code, graphics, music, sounds), unmodified and hash-pinned	the game every brain is trained and measured on	yes
`games/mario_bugged/`	a copy of the clone with the 6 declared benchmark bugs	the bug-detection target, picked in the dashboard	yes
`checkpoints_qa/`	the final 16M brain's closure record, evidence, and the 6.03M anchor checkpoint	dashboard approval check, retraining with anchor-KL, `artifacts.json`	only `final_objective2_16000000/FINAL_OBJECTIVE2.json`
`exploration_data/`	measured data: reachability mask, jump arcs, observed reach, coverage bootstrap, anchor states	QA reward, coverage, anchor-KL training	only `reachable_mask.npz`
`generated/`	everything written as the project runs: incidents/ (bug evidence), run_reports/ (clean runs), coverage_audits/, calibration_runs/, logs/ (training), and cache/ (Python's bytecode, ruff, mypy, pytest, coverage); no __pycache__ is written next to the source	the dashboard, the tools, training, the quality gate	no
`venv_gpu/`	the Python environment: Python 3.12 and every installed library (about 4.5 GB)	every command	no
`.git/`	the repository's full history	git	(git's own folder)

Files in the project root

File	What it is and what it holds	Where it is used	In git
`app.py`	the dashboard server (Flask + Socket.IO): page, REST API (`/api/status`, `/api/project`, `/api/incidents`, `/api/runs`), report and evidence downloads, socket events	started by `python app.py` or `run_dashboard.bat`	yes

File	What it is and what it holds	Where it is used	In git
`custom_mario_env.py`	the game as a Gymnasium environment: actions, observations, info, built-in detectors, the game window's life cycle	training, dashboard, evaluation, tests, tools	yes
`agent_logic.py`	`GlitchHunterWrapper`: coverage recording, the episode lifecycle and the reward	training, dashboard, evaluation	yes
`train_agent.py`	training for Objectives 1 and 2, with safe resume	run by hand (`python train_agent.py`)	yes
`run_dashboard.bat`	Windows double-click launcher for the dashboard	the user	yes
`setup.bat`	Windows one-click setup: finds (or installs with winget) Python 3.12, creates venv_gpu, installs the CPU build of PyTorch (gpu: the CUDA build) and the libraries, checks the brain files, then starts the dashboard	a new user, once after cloning; CI (`setup.bat norun`)	yes
`setup.sh`	the same one-command setup for Linux and macOS	`bash setup.sh`; CI (`bash setup.sh norun`)	yes
`mario_brain_checkpoint.zip`	the 6M brain (Objective 1), 20 MB: the fallback brain	dashboard when the 16M brain is absent; baseline for evaluation	no (git-ignored since Sep 29, 2026; earlier commits hold it)
`glitch_hunter_main_brain.zip`	the final 16M brain (Objectives 2-3), 20 MB, hash-checked	dashboard's default brain; a resume candidate for training	yes (also a release asset that `tools/final_brain.py` installs)
`glitch_hunter_main_brain_coverage.npz`	the 16M brain's coverage map (where it has been in the level)	dashboard's QA reward, coverage maps	yes
`artifacts.json`	SHA-256 of every protected brain, mask, baseline and evidence file	`tools/verify_artifacts.py`, `tools/final_brain.py`, tests	yes
`pyproject.toml`	dependencies and the settings for ruff, mypy, pytest and coverage	`uv sync`, every quality tool, CI	yes
`uv.lock`	exact locked versions of every dependency	`uv sync --active`	yes

File	What it is and what it holds	Where it is used	In git
`requirements.txt`	the same dependencies for plain pip, with the install steps	`pip install -r requirements.txt`, CI	yes
`README.md`	the main guide: overview, installation (one-click setup first), how to run, results	readers, GitHub front page	yes
`LICENSE`	all rights reserved: no use, copying or sharing without the author's written permission; also lists what it does not cover	readers	yes
`.gitignore`	what git must never commit, and why	git	yes
`.gitattributes`	line-ending and binary rules	git	yes
`.pre-commit-config.yaml`	optional checks run on every commit (ruff, mypy, file hygiene)	`pre-commit`	yes

Core source packages

All tracked in git; each folder's `__init__.py` marks it as a Python package and says what it is for.

File	What it is and what it holds	Where it is used
`common/cli.py`	the start-up every tool shares: project root on the path, headless SDL, logging	every script in `tools/`
`common/fileio.py`	atomic file writes, SHA-256 hashes, read-only protection for artifacts	training, reporting, tools
`common/logging_setup.py`	one logging configuration and the progress log	every entry point
`rewards/legacy.py`	the completion reward, unchanged from the 6M training run	`agent_logic.py` in completion mode
`rewards/qa.py`	the QA exploration reward: new coverage, phase-gated completion bonus	`agent_logic.py` in QA mode
`rewards/shared.py`	state and shaping both rewards share, incl. the adaptive death memory	both reward modes
`training/callbacks.py`	the training callbacks: checkpoints with coverage, anchor-KL, stagnation watch, telemetry	`train_agent.py`
`training/checkpoints.py`	reads a checkpoint's step count without loading it; retention verdicts for resume	`train_agent.py`
`training/value_head.py`	the one-off critic reset when QA training starts from the 6M brain	`train_agent.py`

File	What it is and what it holds	Where it is used
`evaluation/completion.py`	the frozen 500-episode test: can a brain still finish Level 1-1?	`tools/evaluate_completion.py`, resume checks
`evaluation/level1_verification.py`	decides whether a Level-1 snapshot is the final brain	`tools/verify_level1.py`
`evaluation/completion_baseline_6M.json`	the 6M brain's frozen completion result (46.8%) that others are compared with	`evaluation/completion.py`; hash-protected
`exploration/config.py`	every tuned value (rewards, thresholds, paths), each with the measurement behind it	nearly every module
`exploration/coverage.py`	the world-space coverage bitmap, shared by all training workers	reward wrapper, training, dashboard, tools
`exploration/reachability.py`	the testable-pixel mask: which pixels Mario can physically reach	coverage percentages, `tools/build_reachability.py`
`exploration/lifecycle.py`	the episode lifecycle: EXPLORE to COMPLETE, and how an episode ends	reward wrapper, dashboard
`exploration/level_completion.py`	when Level 1 counts as fully covered; coverage maps and region lists	training, `tools/remaining_coverage_map.py`

`evaluation/results/` (not in git) holds one completion-test result per evaluated brain; the 16M brain's result is hash-protected and read when choosing what to resume.

reporting/ (Objective 3)

All tracked in git.

File	What it is and what it holds	Where it is used
`__init__.py`	package marker: turns a detected anomaly into a reviewable incident	-
`events.py`	the `Detection` a detector produces, and the synthetic test probe	environment, pipeline
`collision_invariants.py`	the generic bug detectors: clipping into solids, blocked by nothing, hit or stomp without contact, impossible jump	`custom_mario_env.py`
`level_design.py`	loads Level 1-1 as designed, for the detectors to compare against	`collision_invariants.py`
`level1_design.json`	the drawn ground, pipes and steps (37 solids) measured from the clean game	`level_design.py`; rebuilt only by `tools/build_level_design.py`

File	What it is and what it holds	Where it is used
`pipeline.py`	capture, then GIF, reports and replay in the background; duplicate counting; session view	dashboard backend
`evidence.py`	writes the raw evidence: trigger frame, context frames, trajectory, the incident record	`pipeline.py`
`schema.py`	the incident record's format, ids and validation	pipeline, store, tools
`store.py`	the incident folders on disk: atomic, read-only, never overwritten; safe file lookup	pipeline, `app.py`, `tools/incidents.py`
`analysis.py`	severity, detector confidence and the duplicate fingerprint	`pipeline.py`
`render.py`	an incident's GIF, Markdown report and PDF report	`pipeline.py`, `tools/incidents.py`
`fix_hint.py`	an incident's "Where to fix it": the lines in the game's code that cause it and the exact edit for each (delete / change to), worked out from the incident's record and the game's source (read with ast); a lead, not a proven cause	`render.py`, `pipeline.py` (the dashboard's bug card)
`reproduce.py`	replays an incident's episode in a separate process and says whether it happened again	`pipeline.py`
`provenance.py`	which brain, game and code produced an incident	dashboard backend, pipeline
`variants.py`	the clean and bugged game variants, their tree hashes and declared bugs	environment, provenance, tests
`run_report.py`	the clean game's "No Bugs Found" run report: record, final frame, GIF, Markdown, PDF; the list of every saved run (Bug History, `/api/runs`)	dashboard backend, `app.py`

dashboard/

All tracked in git. Everything behind the dashboard page, in one package; `app.py` at the project root serves it, and the page itself is in `dashboard/web/` (next section).

File	What it is and what it holds	Where it is used
`__init__.py`	marks the package; imports nothing, so the game env can use `game_window` without loading the server	Python
`service.py`	the single game thread: Start / Pause / Reset / game switch, pause on bugs, stop at a clean run's end, step pacing, the live status sent to the page	`app.py`

File	What it is and what it holds	Where it is used
`backend.py`	loads the approved brain, runs the game, streams frames and the live status, picks each move (a new route every run; a stuck run ended early), captures incidents, writes run reports	`dashboard/service.py`, `app.py`
`facts.py`	the Overview page's facts, read from the project's own files: the objectives' results, detectors, benchmark bugs and evidence on disk (its API also reports code size, commits and the test count, which the page does not show)	`app.py` (`/api/project`)
`desktop.py`	`run_dashboard.bat`'s helpers: full speed on battery (power mode, no throttling), centred terminal and browser, and Esc in the terminal stops the dashboard like Ctrl+C; run as `python -m dashboard.desktop`	`run_dashboard.bat`, `app.py --desktop`
`game_window.py`	Windows window calls for the game window: centre, show, hide, minimise to the taskbar	`custom_mario_env.py`, `dashboard/desktop.py`

Dashboard page, images, docs and CI

All tracked in git.

File	What it is and what it holds	Where it is used
`dashboard/web/templates/index.html`	the dashboard page: the four tabs, game chooser, controls, video, Live status with its five-step progress list, Bug Tracker, Bug History, detail windows	served by `app.py` at `/`
`dashboard/web/static/js/main.js`	the page's behaviour: tabs, controls, live video, live status, Bug found and run-result cards, Bug Tracker, Bug History, detail windows, reset (also on refresh), game switch	loaded by `index.html`
`dashboard/web/static/css/style.css`	the page's retro arcade theme, status colours and layout	loaded by `index.html`
`dashboard/web/static/vendor/socket.io.min.js`	the Socket.IO browser client (v4.7.2), bundled so the page works offline	loaded by `index.html`
`dashboard/web/static/vendor/fonts/`	the Press Start 2P pixel font and its license (OFL.txt), bundled so the page works offline	loaded by `style.css`
`docs/assets/ui-overview.png`	screenshot of the Overview tab; also the image at the top of the README	`README.md`
`docs/assets/ui-live-testing-ready.png`	screenshot of the Live Testing tab before a test starts	`README.md`
`docs/assets/ui-live-testing-running.png`	screenshot of the Live Testing tab while testing the clean game	`README.md`

File	What it is and what it holds	Where it is used
`docs/assets/ui-bug-found.png`	screenshot of testing stopped on a bug (an impossible jump, bugged game)	`README.md`
`docs/assets/ui-how-it-works.png`	screenshot of the How It Works tab	`README.md`
`docs/assets/ui-bug-history.png`	screenshot of the Bug History tab	`README.md`
`docs/OBJECTIVE1.md`	Objective 1: the 6M brain, how it was trained, its results	readers; `README.md`
`docs/OBJECTIVE2.md`	Objective 2: QA exploration, audits, the final 16M brain	readers; `README.md`
`docs/OBJECTIVE3.md`	Objective 3: detectors, reporting pipeline, benchmark bugs, run reports	readers; `README.md`
`docs/ARCHITECTURE.md`	components, invariants, protected artifacts, where to change what	readers; `README.md`
`docs/PERFORMANCE.md`	time and memory per training worker, and how to measure them	readers; `README.md`
`docs/CONTRIBUTING.md`	how to develop: set-up, quality gate, code rules, commits, releasing the brain	contributors; `README.md`
`docs/SECURITY.md`	what the dashboard exposes, what run_dashboard.bat and the setup scripts change, dependency audit	readers; `README.md`
`docs/THIRD_PARTY_NOTICES.md`	terms of the Mario clone, Nintendo assets and every library	readers; `LICENSE`, `README.md`
`docs/Glitch_Hunter_Tech_Stack.pdf`	this document as a PDF	readers; `README.md`
`docs/Glitch_Hunter_Panel_QA.pdf`	154 questions a presentation panel may ask about the project, with simple-English answers (purpose, AI, the three objectives, the dashboard and its Start testing code path, making it generic, related work, limits)	presenters, before a viva or review
`.github/workflows/ci.yml`	continuous integration: lint, types and tests on Ubuntu, Windows and macOS; the one-click setup on Ubuntu, Windows and macOS; a browser test of the dashboard	GitHub Actions on every push

tools/ (command-line tools)

All tracked in git; each is run by hand as `python tools/<name>.py`, and `tests/test_tools_cli.py` checks every one starts and documents itself.

File	What it does	What it writes
`README.md`	the guide to every tool below	-
`check.py`	every quality gate CI runs: ruff, mypy, tests, artifact hashes (`--full`: slow tests + coverage floor)	nothing
`check_environment.py`	tries to load every library the dashboard needs and, when one will not (for example Windows' Smart App Control blocking PyTorch), says in plain words what to do; run by run_dashboard.bat after an error and by both setup scripts	nothing
`final_brain.py`	for a checkout without them, installs the final 16M brain's files from the GitHub Release (verified against `artifacts.json`), or builds that release zip	the four final-brain files
`verify_artifacts.py`	re-hashes every protected artifact against `artifacts.json`	nothing (`--record` rewrites the manifest)
`benchmark_step.py`	milliseconds per agent step, and where the time goes	nothing
`evaluate_completion.py`	can a brain still finish Level 1-1? HEALTHY / WARNING / REGRESSED against the 6M baseline	`evaluation/results/`
`verify_level1.py`	checks a Level-1 snapshot before it counts as the final brain	a verification JSON beside the snapshot
`pycache_hook.py`	puts one start-up line (a .pth file) into venv_gpu, so every Python it starts (a tool, bare pytest, an editor's Run button) keeps its bytecode in generated/cache/pycache/ and never writes __pycache__/ next to the source; refuses outside a venv; run by both setup scripts	one file in venv_gpu's site-packages
`remaining_coverage_map.py`	where the uncovered reachable pixels are, as a picture and a region list	`generated/coverage_audits/`
`incidents.py`	the incident store: list, show, verify, re-render, reproduce, recover	only new versioned files, never overwrites
`validate_incident_pipeline.py`	37-check end-to-end proof of the reporting pipeline on the real dashboard stack	its own store under `generated/incidents/_validation/`

File	What it does	What it writes
`build_level_design.py`	writes Level 1-1's drawn solids from the clean game	`reporting/level1_design.json`
`collect_jump_arcs.py`	records the engine's real jump arcs	`exploration_data/jump_arcs.npz`
`build_reachability.py`	builds the reachable-pixel mask (the coverage denominator)	`exploration_data/reachable_mask.npz`
`build_anchor_set.py`	records the anchor states a healthy brain sees, for anchor-KL training	`exploration_data/anchor_states.npz`
`migrate_coverage.py`	re-stamps a coverage file to a rebuilt mask, losslessly	a new coverage file
`bootstrap_coverage.py`	seeds the coverage map with the 6M brain's usual routes	`exploration_data/coverage_bootstrap_*.npz`
`calibrate_reward.py`	solves the QA reward's scale against the completion reward	a value in `exploration/config.py`
`calibrate_phase_reward.py`	measures every QA reward channel on real runs (source of the phase-gated values)	`generated/calibration_runs/`
`objective2_evidence/jump_physics.py`	engine ground truth for the 172 px obstacles (tall pipes, staircase top)	nothing
`objective2_evidence/runup.py`	the minimum run-up for a full-height jump	nothing
`objective2_evidence/well_escape.py`	exhaustive search: is the staircase well a soft-lock?	nothing
`objective2_evidence/well_trace.py`	replays one well-escape attempt frame by frame	nothing

tests/

All tracked in git and run by `pytest` (through `tools/check.py` and CI). The first eight are helpers, not tests.

File	What it holds or checks
`conftest.py`	shared fixtures (the real game environment, temporary folders) for every test
`incident_helpers.py`	builds realistic detections and capture context without running the game, and a stand-in brain; used by the incident and dashboard tests

File	What it holds or checks
`mp_workers.py`	worker processes for the multi-process coverage test
`bug_probe.py`	plays every injected-bug scenario on one game variant; run by `test_injected_bugs.py`
`variant_probe.py`	plays one game variant and prints its fingerprint; run by `test_game_variants.py`
`bug_incident_run.py`	one dashboard episode through the real reporting pipeline; run by `test_bug_incidents.py`
`fix_hint_incidents.json`	the eight real incidents of the benchmark bugs, trimmed; read by `test_fix_hint.py`
`fix_probe.py`	plays the bug scenarios on a patched copy of the bugged game; run by `test_fix_hint.py`
`test_arc_reach.py`	reachability from measured jump arcs instead of a rectangle
`test_artifacts.py`	protected files are still the recorded bytes; the main brain is the approved one
`test_bug_incidents.py`	all six benchmark bugs become reproduced, reported incidents; the clean game reports none
`test_collision_invariants.py`	the collision and jump detectors, with a regression test per false alarm
`test_common.py`	`common/` : atomic file I/O, logging, tool start-up
`test_completion_eval.py`	the 500-episode completion evaluation
`test_completion_actor.py`	the evaluation's action choice, checkpoint record and interval maths, on stand-ins
`test_concurrency.py`	thread-safety of the dashboard server
`test_coverage.py`	coverage recording, saving, frontier, adaptive target
`test_coverage_env.py`	coverage from the real game is in world space
`test_coverage_multiproc.py`	coverage is shared across real OS processes
`test_dashboard_backend.py`	what the browser is sent, the live status, the brain's facts, `/api/project`, clean-run endings and reports, a new route every run (the bugged game's first keeps the fixed one), stuck runs ended early, refresh = reset, `--desktop` mode
`test_dashboard_control.py`	the game thread and window: Start, Pause, X, Reset, game switch, pacing, minimised start, the live status sent every few steps
`test_dashboard_facts.py`	the Overview's facts: read from the project's own files, never guessed; the test count
`test_dashboard_e2e.py`	the dashboard in a real browser: Start testing, a bug stops it, the card says where and what to fix (skips without Playwright)

File	What it holds or checks
`test_dashboard_incidents.py`	the dashboard side of Objective 3: stops, Bug Tracker, downloads
`test_dashboard_stop.py`	Esc stops the dashboard like Ctrl+C: in its terminal, and on the page (Esc, then Yes) through /api/stop; another website cannot stop it
`test_desktop.py`	`desktop.py`: power-mode guard, restorer, centred windows
`test_env.py`	the environment's contract and past-bug regressions
`test_episode_lifecycle.py`	the engine timer, EXPLORE to COMPLETE, and how episodes end
`test_check_environment.py`	a library that will not load gets a plain answer (Windows blocking PyTorch, a missing package); the scripts run the check
`test_final_brain.py`	`tools/final_brain.py` installs the 16M brain intact
`test_fix_hint.py`	"Where to fix it" lands on the injected code for every benchmark bug, without reading the bug list, and applying its edits makes every bug's spot play exactly like the clean game
`test_game_variants.py`	the clean game is untouched; the bugged game differs only where declared
`test_game_window.py`	show, hide and minimise calls of the game window
`test_glitch_detection.py`	the built-in detectors behind the Bug Tracker
`test_hud_timer.py`	the on-screen TIME box during long QA episodes
`test_incident_capture.py`	capturing evidence from the real game
`test_incident_pipeline.py`	schema, store, duplicates, reports, without the game
`test_incident_reproduce.py`	replay is honest both ways (reproduced / not reproduced)
`test_incident_tools.py`	`tools/incidents.py` and the end-to-end validation
`test_injected_bugs.py`	each of the six bugs, proved against the clean game
`test_level1_compliance.py`	what surrounds Level-1 completion
`test_level_completion.py`	QA training ends on Level-1 coverage, not a step count
`test_migrate_coverage.py`	`tools/migrate_coverage.py`
`test_noncoverage_taxonomy.py`	the two false-coverage causes found during Objective 2
`test_objective2_protection.py`	Objective 3 never writes into Objective 2's files
`test_phase_reward.py`	the phase-gated QA reward
`test_pycache_hook.py`	a Python with the hook writes no __pycache__ next to the source; the setup scripts install it first; no tool sends bytecode into tools/
`test_qa_resume.py`	QA checkpoints, resuming, and the 6M brain's safety

File	What it holds or checks
`test_quality_gate.py`	the coverage floor is measured to two decimals
`test_reward_calibration.py`	the calibrated reward and its bookkeeping
`test_reward_qa.py`	reward shaping in QA mode
`test_reward_telemetry.py`	the reward telemetry written during training
`test_reward_wrapper.py`	reward shaping in completion (legacy) mode
`test_rewards_death_memory.py`	the adaptive death memory
`test_run_report.py`	the "No Bugs Found" run report, its files and downloads, the list of saved runs and `/api/runs`
`test_setup_scripts.py`	`setup.bat` and `setup.sh` install the pinned torch, check every brain file, and keep their line endings
`test_testable_mask.py`	the coverage denominator never exceeds the real world
`test_tools_cli.py`	every tool starts, parses its options and documents itself
`test_train_agent_stages.py`	`train_agent.py`: resume, device, callbacks, banner
`test_training_callbacks.py`	the training callbacks and checkpoint helpers
`test_value_head.py`	the one-off value-head reset touches only the critic's output layer, on a tiny stand-in

The game: games/mario_clean/ and games/mario_bugged/

Both folders, side by side in `games/`, hold the same third-party Mario clone and are tracked in git. `games/mario_clean/` is never edited (its whole tree is pinned by SHA-256); `games/mario_bugged/` differs only where `INJECTED_BUGS.json` declares a bug. The game code is loaded by `custom_mario_env.py`.

File (in each folder)	What it is and what it holds
`VARIANT.md`	what this copy is for; in `mario_bugged/`, the six bugs and the rules for changing it
`INJECTED_BUGS.json` (`mario_bugged/` only)	the declaration of every benchmark bug: where, which lines, the pinned diff hash
`data/__init__.py`, `data/components/__init__.py`, `data/states/__init__.py`	package markers
`data/setup.py`	starts pygame, opens the game window, loads every image, song and sound
`data/tools.py`	`Control`, the game's main loop and state machine; the image/music/sound loaders
`data/constants.py`	screen size, colours, physics values, state names

File (in each folder)	What it is and what it holds
`data/game_sound.py`	which music and sound effects play when
`data/states/level1.py`	Level 1-1 itself: builds ground, pipes, steps, bricks and enemies, and runs every frame's collisions (in `mario_bugged/`, carries five of the six bugs: stair, pipe and ceiling clipping, invisible wall, stomp from too far)
`data/components/mario.py`	Mario: movement, jumping, growing, dying, animation (in `mario_bugged/`, carries the open-sky jump bug)
`data/components/enemies.py`	Goombas and Koopas
`data/components/bricks.py`	bricks and the pieces they break into
`data/components/coin_box.py`	? boxes
`data/components/coin.py`, `data/components/flashing_coin.py`	coins that pop out of boxes; the blinking coin in the score bar
`data/components/powerups.py`	mushroom, 1-up, fire flower, star, fireball
`data/components/collider.py`	invisible solid rectangles: ground, pipes, steps
`data/components/checkpoint.py`	invisible triggers that release enemies and mark events
`data/components/flagpole.py`, `data/components/castle_flag.py`	the flagpole at the end and the castle flag
`data/components/info.py`	the score bar: score, coins, world, time
`data/components/score.py`	the floating score numbers
`resources/graphics/level_1.png`	the level background
`resources/graphics/mario_bros.png`	Mario's sprites
`resources/graphics/smb_enemies_sheet.png`	enemy sprites
`resources/graphics/tile_set.png`	bricks, boxes and tiles
`resources/graphics/item_objects.png`	coins, power-ups, flag
`resources/graphics/text_images.png`	the score bar's font
`resources/music/`	8 tracks: `main_theme`, `main_theme_sped_up`, `invincible`, `game_over` (.ogg); `death`, `flagpole`, `out_of_time`, `stage_clear` (.wav)
`resources/sound/`	13 effects (.ogg): `big_jump`, `small_jump`, `brick_smash`, `bump`, `coin`, `count_down`, `fireball`, `kick`, `one_up`, `pipe`, `powerup`, `powerup_appears`, `stomp`

The images and sounds are Nintendo's (see `docs/THIRD_PARTY_NOTICES.md`). No `__pycache__/` is written for the game: bytecode goes to generated/cache/pycache/ (tools/pycache_hook.py, `common/__init__.py`).

Brains, measured data and training evidence (mostly not in git)

Mostly too large for git; the important ones are hash-pinned in `artifacts.json` and checked by `tools/verify_artifacts.py`. Only the final brain's four files are in git: `FINAL_OBJECTIVE2.json` and `reachable_mask.npz` below, and the two root files above. `tools/final_brain.py` can also install them from the GitHub Release.

File	What it is and what it holds	Where it is used
`checkpoints_qa/final_objective2_16000000/FINAL_OBJECTIVE2.json`	Objective 2's closure record: the approved 16M brain's hash and results (in git)	the dashboard approves the brain against it
`…/evidence/train.log`	the final training run's log	evidence of how the 16M brain was made
`…/evidence/console.log`	the final training run's console output	evidence
`…/evidence/reward_telemetry.jsonl`	every reward channel, per episode, for the whole run (17 MB)	evidence; reward analysis
`…/evidence/coverage_audit_trail.jsonl`	coverage at every checkpoint of the run	evidence; coverage history
`…/evidence/glitch_hunter_qa_16000000_steps_d906d09e.json`	the 16M brain's 500-episode completion result (56.6%)	evidence
`…/evidence/completion_baseline_6M.json`	the 6M baseline as it stood at closure	evidence
`checkpoints_qa/pre_main_6032768/glitch_hunter_qa.zip`, `…_coverage.npz`	the 6.03M "healthy" checkpoint: the reference brain for anchor-KL	loaded by anchor-KL training (the final run used it)
`checkpoints_qa/pre_main_6032768/reward_telemetry.jsonl`, `coverage_audit_trail.jsonl`	that checkpoint's own telemetry and coverage trail	record only
`exploration_data/reachable_mask.npz`	the reachable-pixel mask: the denominator of every coverage % (in git)	coverage, QA reward, tools
`exploration_data/jump_arcs.npz`	the engine's real jump arcs (228 arcs)	building the mask
`exploration_data/observed_reach.npz`	places real play reached that the arc model had to keep	building the mask
`exploration_data/coverage_bootstrap_6000000_mask_v4.npz`	the 6M brain's routes, seeding the QA coverage map	QA training start

File	What it is and what it holds	Where it is used
`exploration_data/anchor_states.npz`	6,007 frames the healthy brain sees, where anchor-KL is measured	anchor-KL training
`evaluation/results/glitch_hunter_qa_16000000_steps_d906d09e.json`	the 16M brain's completion verdict	read when choosing what to resume
`generated/coverage_audits/glitch_hunter_main_brain_coverage_remaining_map.png`	the 16M brain's coverage map: red = reachable but never visited (84.25% covered)	reading only
`generated/coverage_audits/glitch_hunter_main_brain_coverage_remaining_regions.json`	the uncovered regions of that map, with sizes and positions	reading only

Runtime output: generated/ (not in git)

Everything the project writes as it runs goes into `generated/`. The dashboard writes `generated/incidents/` and `generated/run_reports/` while it tests; they stay on this machine and are never overwritten; a report goes only when you delete it on the Bug History page. The tools write `generated/coverage_audits/` and `generated/calibration_runs/`, training writes `generated/logs/`, and Python's bytecode, ruff, mypy, pytest and coverage keep their caches in `generated/cache/`.

File	What it is and what it holds	Where it is used
`generated/incidents/INC-<date>-<time>-<id>/`	one folder per bug found, read-only once written	Bug Tracker, Bug History, `/incidents/...` downloads, `tools/incidents.py`
`…/incident.json`	the full record: detector, measurements, position, game and brain identity	reports, Bug Tracker
`…/trigger.png`	the exact frame the bug was detected, full resolution	reports, "Trigger frame" link
`…/context_frames.zip`	the stream frames just before the trigger	the GIF, the PDF film strip
`…/trajectory.json`	Mario's state and the action taken, frame by frame, before the trigger	the reports; replay
`…/context.gif`	the moments before the bug, then the trigger frame held	"GIF" link
`…/report.md`, `…/report.pdf`	the bug report in Markdown and PDF	"Markdown / PDF report" links
`…/reproduction.json`	the replay's verdict: reproduced or not	reports, Bug Tracker
`…/manifest.json`	SHA-256 of every file, and how each report was produced	`tools/incidents.py verify`

File	What it is and what it holds	Where it is used
`generated/incidents/occurrences.jsonl`	every later sighting of an already-known bug (raises its "Seen" count)	Bug Tracker counts
`generated/incidents/_validation/`	appears only after `tools/validate_incident_pipeline.py`; its own test bugs, hidden from the dashboard	the validation tool
`generated/run_reports/RUN-<date>-<time>-<id>/`	one folder per clean-game run that reached the castle	the Level complete card, Bug Tracker, Bug History
`…/run.json`	the run's record: outcome, length, furthest point, score, bugs (none)	the reports
`…/final.png`	the last frame of the run, full resolution	"Final frame" link
`…/finish.gif`	the run's last seconds	"GIF" link
`…/report.md`, `…/report.pdf`	the "No Bugs Found" report in Markdown and PDF	report links, "Download all (.zip)"

Environment and caches (not in git)

All rebuilt automatically or by the install steps; described as folders because they hold thousands of generated files.

Folder or file	What it is and what it holds	Where it is used
`venv_gpu/`	the project's Python 3.12.10 environment, about 26,500 files and 4.5 GB	every command, `run_dashboard.bat`
`venv_gpu/Lib/site-packages/`	the 58 installed packages: PyTorch 2.14.0 + CUDA 12.6 (4.0 GB of it), Stable-Baselines3, pygame, Flask, NumPy, OpenCV, fpdf2, pytest, ruff, mypy and their dependencies	imported by all project code
`venv_gpu/Scripts/`	`python.exe`, `pythonw.exe` (no console, used for background helpers), `pip`, `pytest`, `ruff`, `mypy`, `pre-commit` and activation scripts	every command
`venv_gpu/pyvenv.cfg`, `Include/`, `share/`	the environment's settings (which base Python) and package extras	Python itself
`.pytest_cache/`	pytest's cache (last failures); appears when pytest runs without `-p no:cacheprovider`	`pytest`
`.git/`	the full version history and GitHub link (about 80 MB)	git

Notes

- **torch is pinned to 2.14.0** (CUDA 12.6 build): it has no published security advisories, and it meets Stable-Baselines3's own torch requirement, so SB3 installs normally. It replaced 2.5.1 (CUDA 12.1) on

2026-09-25; both brains play move-for-move the same on it. Install torch first (so Windows gets the GPU build), then `requirements.txt` — or simply `uv sync --active`.

- **eventlet must never be installed.** Flask-SocketIO picks its async mode from what is installed; the dashboard runs in plain threading mode on purpose.
- **Headless everywhere except the dashboard:** training, evaluation, tests and incident replays use the SDL dummy video driver; only the dashboard opens a real game window.
- **Artifacts are protected by hash:** most brains, masks and coverage files are large and git-ignored (only the final 16M brain's four files are in git); `artifacts.json` (tracked) holds their SHA-256 values.